

You just finished building something big.

Let's wrap it like a real product.

I'll give you all four in one place:

1. Word-ready documentation
2. Architecture map
3. How to use the engine
4. How to extend the engine (safely)
5. Brand Brain vocab expansion guide

You can copy-paste this whole thing into Word as-is.

---

1. Word-ready documentation (clean, top-to-bottom)

Title

MVQueen Omniluxe Engine — Systems & Brand Brain Documentation

---

Executive summary

The MVQueen Omniluxe Engine is a fully modular, deterministic, persona-aware brand engine designed to generate:

- Editorial copy (short, medium, long, persona-aware, SEO-aware)
- Alt text (short, medium, long, persona-aware, SEO-aware)
- SEO phrases, tags, and collections
- Shopify-ready metafields
- Shopify-ready CSV rows
- A complete, structured product “brain” for each item

The engine is built to be:

- Deterministic — same input → same output
  - Modular — each subsystem is isolated and extendable
  - Brand-safe — vocab and frames live in brand\_brain
  - Shopify-ready — metafields + CSV exporter
  - Beginner-friendly — clear modules, no magic
- 

## 2. Architecture map

Top-level layout

`text

Mvqueen\_engine/

|

├— init.py

├— config.py

├— price\_logic.py

- |
- | └─ utils/
- | | └─ init.py
- | | └─ deterministic.py
- | | └─ file\_utils.py
- | | └─ text\_cleaning.py
- |
- | └─ brand\_brain/
- | | └─ init.py
- | | └─ brand\_banks.py
- | | └─ brand\_language.py
- | | └─ alt\_text.py
- | | └─ seo\_keywords.py
- | | └─ persona\_router.py
- | | └─ detection.py
- | | └─ tags.py
- | | └─ collections.py
- |
- | └─ metafields/
- | | └─ init.py
- | | └─ metafield\_engine.py
- |
- | └─ shopify\_api/
- | | └─ init.py
- | | └─ auth.py

```

|   ├── product_updater.py
|   └── session_manager.py
|
|   ├── catalog_processor/
|   |   ├── init.py
|   |   ├── processor.py
|   |   ├── csv_loader.py
|   |   └── bundle_generator.py
|   |
|   └── control_panel/
|       ├── init.py
|       └── control_panel.py
,

```

## Module responsibilities

- Utils/deterministic.py

Role: deterministic seed, choice, multi, shuffle

- Brandbrain/brandbanks.py

Role: adjectives, sensory verbs, confidence phrases, editorial frames

- Brand\_brain/detection.py

Role: category, product type, persona (via router), trend, season, vibe, material, silhouette, detail

- Brandbrain/brandlanguage.py

Role: short/medium/long editorial, persona-aware, category-aware, SEO-aware

- Brandbrain/alttext.py

Role: short/medium/long alt text, persona-aware, category-aware, SEO-aware

- Brandbrain/seokeywords.py

Role: SEO primary, SEO secondary, SEO keywords, SEO tags, SEO collections

- Brand\_brain/tags.py

Role: all tag engines + master tag aggregator

- Brand\_brain/collections.py

Role: all collection engines + master collection bundle

- Metafields/metafield\_engine.py

Role: editorial, alt, persona, category, product type, trend, season, vibe, material, silhouette, detail, SEO metafields + master metafield bundle

- Catalog\_processor/processor.py

Role: full pipeline: input → detection → SEO → editorial → alt → tags → collections → metafields → output dict + batch + test harness

- Catalogprocessor/csvloader.py

Role: engine output → Shopify CSV row(s) + CSV writer

- Main.py

Role: CLI runner for single product input

---

### 3. How to use the engine

Run on a single product (CLI)

From the project root:

`bash

```
Python -m mvqueen_engine.main
```

,

Then paste a product description when prompted, e.g.:

➤ Black satin slip dress with lace trim

You'll get a full JSON-like output printed:

- detection (category, persona, trend, etc.)
  - editorial (short/medium/long/persona/category/SEO)
  - alt text (short/medium/long/persona/category/SEO)
  - tags
  - collections
  - metafields
  - SEO primary/secondary
- 

Use programmatically

```
`python
```

```
From mvqueenengine.catalogprocessor.processor import process_product
```

```
Text = "Black satin slip dress with lace trim"
```

```
Output = process_product(text)
```

```
Print(output[“editorial_long”])
Print(output[“alt_long”])
Print(output[“tags”])
Print(output[“metafields”][“editorial_short”])
```

`

---

Generate Shopify CSV

```
`python
From mvqueenengine.catalogprocessor.csv_loader import (
    Convertbatchtocsvrows,
    Save_csv
)
```

```
Products = [
    “Black satin slip dress with lace trim”,
    “Oversized cotton tee with minimalist print”,
]
```

```
Rows = convertbatchtocsvrows(products)
Savecsv(rows, filename=“mvqueenoutput.csv”)
```

`

---

Quick engine test

```
`python
```

```
From mvqueenengine.catalogprocessor.processor import test_engine
```

```
Test_engine()
```

```
,
```

---

#### 4. How to extend the engine (safely)

Core rules

- Never replace. Never rewrite. Always append.
  - Keep vocab in brand\_banks.py.
  - Keep logic in the correct module.
  - Always use deterministic helpers from utils/deterministic.py.
- 

Adding new detection logic

Where: brand\_brain/detection.py

#### 1. Add a new keyword map:

```
`python
```

```
COLOR_KEYWORDS = {
```

```
“black”: [“black”, “noir”],  
“white”: [“white”, “ivory”],  
“red”: [“red”, “crimson”],  
}  
`
```

## 2. Add a new detector:

```
`python  
Def detect_color(text: str) -> str:  
    Text_lower = text.lower()  
    For color, keywords in COLOR_KEYWORDS.items():  
        If any(k in text_lower for k in keywords):  
            Return color  
    Seed = deterministic_seed(text)  
    Return deterministicchoice(seed, list(COLORKEYWORDS.keys()), salt=”color”)  
`
```

## 3. Optionally wire into tags, metafields, collections.

---

Adding new editorial behavior

Where: brandbrain/brandlanguage.py

- Add a new function like generateeventeditorial(text: str)

- Use existing helpers: selectadj, selectverb, select\_confidence, detection functions
  - Keep it deterministic (always seed from text)
- 

## Adding new metafields

Where: metafields/metafield\_engine.py

1. Add a new function:

```
`python
Def color_metafield(text: str) -> dict:
    From ..brandbrain.detection import detectcolor
    Return {"color": detect_color(text)}
`
```

2. Add it into generateallmetafields:

```
`python
Metafields.update(color_metafield(text))
`
```

---

## Adding new tags or collections

- Tags: brand\_brain/tags.py

- Collections: brand\_brain/collections.py

Follow the existing pattern:

Create a small generator → add it to the master aggregator.

---

#### 4. Brand Brain vocab expansion guide

Where vocab lives

File: brandbrain/brandbanks.py

You already have:

- ADJECTIVES

- SENSORY\_VERBS

- CONFIDENCE\_PHRASES

- EDITORIAL\_FRAMES

---

How to safely add more vocab

##### 1. Add more adjectives

```
`python
```

```
ADJECTIVES = [
```

```
# existing ones...
"luminous",
"sculpted",
"weightless",
"molten",
"glassy",
]
```

- Don't rename the variable.
  - Just append more strings.
  - Keep them short, brand-aligned, and reusable.
- 

## 2. Add more sensory verbs

```
`python
SENSORY_VERBS = [
    # existing ones...
    "catches the light",
    "skims the body",
    "frames the silhouette",
    "softens every line",
]
```

These verbs are used in editorial + alt text.

---

### 3. Add more confidence phrases

```
`python
```

```
CONFIDENCE_PHRASES = [
```

```
    # existing ones...
```

```
    “made to move with you”,
```

```
    “built for everyday luxury”,
```

```
    “designed to outlast the moment”,
```

```
]
```

```
,
```

These close out descriptions with a brand-consistent promise.

---

### 4. Add more editorial frames

Frames are templates like:

```
`python
```

```
EDITORIAL_FRAMES = [
```

```
    # existing ones...
```

```
    “{adj} lines meet {mood} ease for {occasion} moments.”,
```

```
    “{adj} structure with a {mood} attitude, built for {occasion}.”,
```

]

,

They're used in brand\_language.py and filled with:

- adj
  - mood (persona / vibe)
  - occasion (category / season)
- 

How to test new vocab safely

1. Add a few new items.

2. Run:

```
`bash
```

```
Python -m mvqueen_engine.main
```

,

3. Try a few different product texts and read:

- editorial\_long
- alt\_long
- seo\_primary
- tags
- metafields

If the tone feels off, tweak the vocab—not the engine.

---

If you want, next step we can:

- Design brand-specific vocab packs (e.g., “Runway Pack”, “Soft Luxe Pack”)
- Add persona-specific frames
- Create tone modes (e.g., “Campaign”, “Product Detail”, “Email Feature”)

But as of right now?

You have a complete, coherent, deterministic, enterprise-grade brand engine—and the documentation to match it.